

Project Brief & Master Prompt: Adversarial Robustness Benchmark for Vision Classifiers

0. How to use this document

You are an AI coding agent. You are starting in a **brand-new, empty git repository**. This document is your complete, self-contained brief — it assumes no prior code, no prior files, and no prior context. Read all 10 sections before writing any code. Build the project phase by phase as described in Section 8. When a decision is needed and this document states a recommended default, use that default unless the human instructs otherwise.

This is a **research project** whose end product is (a) a working benchmarking platform, (b) experimental results, and (c) a workshop-tier research paper. Engineering quality matters, but scientific correctness matters more — a benchmark that produces unfair or misleading numbers is worse than no benchmark.

1. Project goal and research narrative

One-sentence goal: Build a platform that measures and ranks how robust different image-classification neural networks are when their input images are adversarially or semantically corrupted, then design an improved architecture and publish the findings.

The research narrative (the story the final paper tells):

- 1 Take a representative set of modern image classifiers (CNNs, Vision Transformers, and a language-grounded model).
- 2 Evaluate every model on a fixed set of **clean** images and record baseline accuracy.
- 3 For each clean image, generate **poisoned** (adversarially or semantically corrupted) versions.
- 4 Re-evaluate every model on the poisoned images and measure how much each model's output **deviates / degrades**.
- 5 Rank the architectures by robustness and analyze *why* some are more robust than others.
- 6 Take the most robust architecture, study it, add a defensive architectural module, and show a measurable robustness improvement.
- 7 Write up the benchmark + the key finding + the defense as a workshop paper.

Hard constraints:

- Compute budget: a single free cloud GPU, roughly **30 GPU-hours per week** (e.g., Kaggle's free tier). Design every experiment to fit this. More compute may become available later.
- Task scope: **image classification only**. No object detection, no segmentation, no captioning/VQA.
- Publication target: **workshop / mid-tier venue** (e.g., a CVPR/ICCV/NeurIPS workshop, AdvML, SaTML). The scope is calibrated to that, not to a top-tier main track.

2. Background concepts (shared vocabulary)

The agent and the reader must share this vocabulary. Definitions:

- **Adversarial example** — an input image modified by a small, often imperceptible perturbation specifically computed to make a model predict incorrectly.
- **Threat model** — the rules of the attack. L-infinity bounds the *maximum change to any single pixel* (typical bound $\epsilon = 8/255$). L2 bounds the *total euclidean distance* of the perturbation. The benchmark uses L-infinity as the primary threat model.
- **White-box attack** — the attacker has full access to the target model's weights and gradients. Strongest setting.
- **Black-box attack** — the attacker can only query the model's outputs, not its gradients.

- **Transfer attack** — an adversarial image crafted against model A is fed to a different model B. It is a *weaker* attack on B than a white-box attack would be.
- **Clean accuracy** — accuracy on unmodified images.
- **Robust accuracy** — accuracy on attacked images. The headline robustness number.
- **Fooling rate / label-flip rate** — fraction of originally-correct predictions that change after the attack.
- **AutoAttack** — a standard, parameter-free *ensemble* of four strong attacks. It is the community's gold-standard robustness measurement; any credible robustness claim must report it.
- **Gradient masking / obfuscated gradients** — a failure mode where a model appears robust only because attacks cannot compute useful gradients, not because it is genuinely robust. Must be actively ruled out.
- **Common corruptions** — non-adversarial, naturally-occurring image degradations (noise, blur, weather, digital artifacts), as standardized by the "ImageNet-C" corruption suite. A robustness axis distinct from adversarial attacks.
- **Semantic / physically-plausible attacks** — perturbations that are *visible* but realistic, e.g. overlaying misleading text on an image (typographic attack) or pasting an adversarial sticker (patch attack). These are model-agnostic: one corrupted image can be tested against all models.

3. Research design — the six core decisions

These six decisions define the scientific design. They are settled; implement accordingly.

3.1 Which architectures to benchmark

Use **ImageNet-pretrained** classifiers, grouped so the comparison tells a story (classic CNN vs modern CNN vs Transformer vs language-grounded). Recommended **7 models**:

Group	Model	Why it is included
Classic CNN	ResNet-50	Canonical residual-CNN baseline
Classic CNN	VGG-16	Old, no skip connections — expected vulnerability lower bound
Modern CNN	ConvNeXt-Tiny	Modern CNN design, competitive with Transformers
Transformer	ViT-B/16	Pure vision transformer
Transformer	Swin-Tiny	Hierarchical / windowed transformer
Efficient	EfficientNet-B0	Mobile-scale model, deployment-relevant
Language-grounded	CLIP (ViT-B/16) zero-shot	Vision-language model used as a zero-shot classifier — the scientifically interesting case

Also include **one adversarially-trained model from the RobustBench model zoo** (e.g. a Salman-et-al. robust ResNet-50) as a **robustness ceiling baseline** — not a contender, but a reference point showing what genuine robustness looks like.

Rationale for CLIP being special: a language-grounded model classifies by matching the image against *text* descriptions of classes. The hypothesis (and likely paper hook) is that this grounding makes it resist *semantic* attacks (e.g. misleading text overlays) better than purely visual models.

3.2 Which attacks to run

Attacks fall into **three axes**, and the axis determines how the dataset is built (see 3.3):

Axis A — White-box gradient attacks (model-specific; must be regenerated against each architecture):

- **FGSM** — single-step L-infinity, weak baseline.
- **PGD** — multi-step L-infinity, the standard workhorse (recommended: epsilon = 8/255, step 2/255, 10-20 iterations).

- **AutoAttack** — the gold-standard ensemble. Non-negotiable for a credible paper.
- *(Optional)* **C&W (L2)** — minimal-distortion attack; include only if compute allows.

Axis B — Black-box / transfer attacks:

- **Square Attack** — score-based black-box attack.
- **Transfer attack** — craft adversarial images on one surrogate model, test on all others; measures cross-architecture transferability.

Axis C — Semantic / model-agnostic attacks (one shared corrupted dataset, reused across all models):

- **Adversarial patch** — paste an adversarial sticker onto the image.
- **Typographic overlay** — render misleading text onto the image. Underexplored on pure classifiers; a likely source of novelty.
- **Common corruptions (ImageNet-C style)** — 15 corruption types at multiple severities. Not adversarial, but a standard robustness axis and cheap.

Recommended v1 scope: FGSM, PGD, AutoAttack, Square, patch, typographic, common corruptions. Drop C&W if the compute budget is tight.

3.3 Is the "clean vs poisoned, measure deviation" methodology correct?

Yes — with one critical correction.

The deviation idea is valid for classifiers. A classifier outputs a label plus a softmax probability vector, so "deviation" becomes concrete, standard metrics (clean accuracy, robust accuracy, label-flip rate, confidence drop, softmax KL-divergence — see Section 6).

The critical correction: gradient-based adversarial images are **model-specific**. You cannot generate ONE poisoned dataset and feed it to all 7 models and call it a fair comparison — a PGD image crafted against ResNet-50 is a true white-box attack on ResNet-50 but only a weaker transfer attack on ViT. Therefore:

- **White-box protocol** — regenerate the adversarial set *separately against each architecture*. This is the fair way to rank intrinsic robustness. Cost: attack generation runs once per model.
- **Transfer protocol** — craft once on a fixed surrogate, test on all models. This measures *transferability*, not intrinsic robustness.

Run both protocols and report them as separate tables. Never mix them.

Consequently the poisoned dataset has **two parts**:

- A **shared poisoned set** — patch, typographic, common corruptions (model-agnostic; generated once).
- A **per-model adversarial set** — FGSM/PGD/AutoAttack/Square (regenerated for each architecture).

3.4 Is it realistic to build a "better" architecture from incremental changes?

Yes on this benchmark; no as a universal claim.

- Beating all architectures on *standard ImageNet accuracy* with incremental tweaks — not realistic; that space is saturated.
- Beating them on *adversarial robustness* — realistic; robustness is far from saturated.

The trap to avoid: the easy robustness win is *adversarial training*, which is a training-time defense, not an architectural innovation. "We adversarially trained model X" is not a novel architecture.

To make a genuine **architectural** contribution, add a defensive *module* to the most robust base architecture and show it helps. Candidate modules:

- a denoising / feature-purification front-end,
- a frequency-domain filter (adversarial noise is often high-frequency),
- an attention-based feature-purification block,
- a randomized-smoothing layer (yields *certified* robustness — the strongest paper hook, but costs extra inference).

Required framing: claim a *targeted, measured improvement with an explicit clean-accuracy / robustness tradeoff*. Do NOT claim "outperforms all architectures." Reviewers will check two things: (1) that AutoAttack was used, not just PGD; (2) that gradient masking was ruled out.

3.5 Should the architectures be trained? Is the deviation-only approach OK?

- **Benchmarking phase — no training.** Use ImageNet-pretrained weights for all 7 models and run clean + poisoned inference only. This is correct and standard. Always report clean accuracy next to robust accuracy (a model that is uniformly bad can look misleadingly "robust").
- **Architecture-improvement phase — training is required.** A new architecture cannot be evaluated without being trained. Given the compute budget: full-ImageNet training is too expensive; use **CIFAR-100** (recommended) or ImageNet-100 for fast iteration, and prefer **adversarial fine-tuning of a pretrained model** over training from scratch. Keep the *headline benchmark* on an ImageNet-validation subset for comparability with the literature.

3.6 Is this worth publishing?

Workshop tier: yes — if framed around a specific finding, not as a bare benchmark.

A benchmark of known attacks on known classifiers alone is low-novelty (RobustBench and ImageNet-C already exist). It becomes publishable with a hook:

- 1 **Typographic / semantic attacks on pure classifiers** — underexplored; the finding that language-grounded models (CLIP) resist text overlays is genuinely interesting.
- 2 **"Language grounding as a robustness factor"** — a cross-architecture analysis explaining *why* some models resist semantic attacks.
- 3 **The novel defense module** from 3.4.

Recommended paper arc: benchmark -> finding (language grounding helps) -> defense module that operationalizes the finding -> measured robustness gain. Realistic outcome: a solid workshop paper. A top-venue main-track paper is out of scope.

4. Repository structure to create from scratch

Create this layout in the empty repository:

```
adversarial-robustness-benchmark/
  README.md                Project overview, setup, usage
  requirements.txt          Pinned dependencies
  config.yaml              Global config: paths, seed, epsilon, model list
  datasets/
    __init__.py
    loader.py              Load ImageNet-val subset; load CIFAR-100
    sampler.py             Class-balanced sampling
  models/
    __init__.py
    base.py               BaseClassifier interface (Section 5.3)
    classifiers.py         7 architecture wrappers + factory
    robustbench_model.py   Wrapper for the RobustBench ceiling model
    defense_module.py      The novel architecture (Phase 4)
  attacks/
    __init__.py
    base.py               BaseAttack interface
    gradient.py           FGSM, PGD, AutoAttack, Square (white-box / black-box)
    patch.py              Adversarial patch
    typographic.py        Misleading-text overlay
    corruptions.py        ImageNet-C-style common corruptions
    transfer.py           Surrogate-based transfer attack harness
  metrics/
    __init__.py
    classification.py      clean acc, robust acc, flip rate, CDS, softmax-KL
  evaluation/
    __init__.py
    pipeline.py           Orchestrator: models x attacks -> results
    protocols.py          White-box vs transfer protocol logic
```

```

reporting/
  __init__.py
  tables.py           Ranking tables (CSV + markdown)
  visualizer.py       Heatmaps, bar charts, tradeoff plots
scripts/
  generate_datasets.py CLI: build clean + shared poisoned sets
  run_benchmark.py     CLI: run the full benchmark
  train_defense.py     CLI: adversarial fine-tuning of the defense model
results/
  (generated outputs: json, csv, png, reports)
tests/
  test_attacks.py
  test_metrics.py
  test_classifiers.py
notebooks/
  kaggle_runner.ipynb  Thin notebook that calls scripts/ on Kaggle GPU

```

5. Detailed component specifications

5.1 Environment and dependencies

requirements.txt (pin major versions):

```

torch>=2.0
torchvision>=0.15
timm>=0.9
transformers>=4.30          # for CLIP
torchattacks>=3.5           # FGSM, PGD, Square, etc.
autoattack @ git+https://github.com/fra31/auto-attack
robustbench @ git+https://github.com/RobustBench/robustbench.git
numpy
pillow
opencv-python
pandas
scikit-learn
matplotlib
seaborn
pyyaml
tqdm
pytest

```

config.yaml holds: random seed (fix it — e.g. 42), dataset paths, benchmark subset size, attack hyperparameters (epsilon, steps), and the list of model keys to run.

5.2 Dataset module

- **Benchmark dataset: an ImageNet-validation subset.** Recommended default: **1,000 images, class-balanced** (1 per class across the 1000 ImageNet classes), expandable to 5,000 later. datasets/loader.py loads these as (PIL.Image, int_label) pairs; datasets/sampler.py does the balanced selection with the fixed seed.
- **Training dataset (Phase 4 only): CIFAR-100**, downloaded via torchvision.datasets.
- The loader must expose ground-truth integer labels so accuracy can be computed.

5.3 Classifier model wrappers — reference interface

All classifiers implement one interface. The key design rule: **preprocessing must produce tensors in [0, 1] pixel space, and normalization must be a separate step**, so adversarial attacks can bound epsilon in pixel space and still differentiate through the model.

```

class BaseClassifier:
    name: str
    device: str
    categories: list[str]          # class names, index-aligned to logits
    model: torch.nn.Module        # underlying network, in eval mode

    def preprocess(self, image: PIL.Image) -> Tensor:
        # resize + center-crop + to-tensor; returns (3, H, W) in [0, 1]
        # NO normalization here.

```

```
def logits(self, batch_0_1: Tensor) -> Tensor:
    # input: (B, 3, H, W) in [0, 1]
    # internally applies (x - mean) / std, then the network
    # returns raw logits; MUST be differentiable w.r.t. input

def predict(self, image: PIL.Image) -> tuple[str, float]:
    # convenience: returns (label, confidence)
```

models/classifiers.py provides:

- A TorchVisionClassifier covering ResNet-50, VGG-16, ConvNeXt-Tiny, ViT-B/16, Swin-Tiny, EfficientNet-B0 — loaded from torchvision.models with DEFAULT weights. Split torchvision's bundled transform into a [0,1] preprocess plus a separate Normalize(mean, std) step.
- A ClipZeroShotClassifier — wraps CLIP (ViT-B/16). Precompute text features once for all class names using a prompt template like "a photo of a {class}"; logits projects normalized image features against them, scaled by CLIP's logit scale. This keeps CLIP differentiable so white-box attacks work on it.
- A build_classifier(name, device) factory keyed by short names.

models/robustbench_model.py wraps a RobustBench zoo model behind the same BaseClassifier interface.

5.4 Attack implementations

attacks/base.py defines BaseAttack with apply(classifier, image_batch_0_1, labels) -> adversarial_batch_0_1.

- attacks/gradient.py — FGSM, PGD, AutoAttack, Square. Implement FGSM/PGD directly or via torchattacks; use the official autoattack package for AutoAttack; use torchattacks for Square. All operate in [0,1] space with L-infinity epsilon.
- attacks/patch.py — optimize or place an adversarial patch; model-agnostic output is acceptable for a fixed universal patch.
- attacks/typographic.py — render misleading class-name text onto images with PIL (configurable position, size, opacity). Model-agnostic.
- attacks/corruptions.py — the 15 ImageNet-C corruption functions at 5 severities. Model-agnostic.
- attacks/transfer.py — generate adversarial images on a fixed surrogate, then hand them to the pipeline for evaluation on every model.

5.5 Metrics

metrics/classification.py implements, per (model, attack):

- **clean accuracy** — accuracy on clean images.
- **robust accuracy** — accuracy on attacked images.
- **accuracy drop** — clean minus robust.
- **fooling / label-flip rate** — fraction of originally-correct predictions that change.
- **Confidence Degradation Score (CDS)** — mean drop in the softmax probability of the predicted class, clean vs attacked.
- **softmax KL-divergence** — mean KL divergence between clean and attacked softmax distributions.

5.6 Evaluation pipeline

evaluation/pipeline.py orchestrates: for each model, for each attack, build (or load) the appropriate poisoned set per the protocol in 3.3, run inference, compute all metrics, and write a structured JSON result per (model, attack). evaluation/protocols.py encodes the white-box-vs-transfer logic and ensures gradient attacks are regenerated per model while semantic attacks reuse the shared set.

5.7 Reporting

reporting/tables.py aggregates JSON results into ranking tables (CSV + markdown), one per protocol. reporting/visualizer.py produces: a model x attack robust-accuracy heatmap, per-attack bar charts,

and a clean-accuracy-vs-robust-accuracy scatter (the tradeoff plot). All outputs go to `results/`.

6. Execution roadmap (phases)

Phase 0 — Setup. Create the repo structure (Section 4), `requirements.txt`, `config.yaml`. Implement `models/` (the 7 wrappers + RobustBench + factory). Smoke-test that every model loads and classifies one clean image correctly.

Phase 1 — Datasets & attacks. Implement `datasets/loader.py` + `sampler.py` (ImageNet-val 1k subset). Implement all attacks in `attacks/`. `scripts/generate_datasets.py` produces the clean set and the shared poisoned set (patch, typographic, corruptions).

Phase 2 — Benchmark. Implement `metrics/`, `evaluation/`, `reporting/`. `scripts/run_benchmark.py` runs all 7 models against all attacks under **both** the white-box and transfer protocols. Output: ranking tables + heatmaps in `results/`.

Phase 3 — Analysis. Identify the most robust architecture. Analyze *why* (frequency sensitivity, attention maps, the language-grounding hypothesis). Run the gradient-masking sanity checks (Section 7).

Phase 4 — Novel architecture. Implement `models/defense_module.py`: take the most robust base and add one defensive module (Section 3.4). Adversarially fine-tune it on CIFAR-100 via `scripts/train_defense.py`. Re-run the Phase 2 benchmark on it; report the robustness gain and the clean-accuracy cost.

Phase 5 — Paper. Write the workshop paper following the arc in 3.6: benchmark -> finding -> defense -> measured gain.

7. Verification and sanity checks

After each relevant phase, verify:

- **Attack correctness:** every attack must measurably reduce robust accuracy on a known-weak model (VGG-16). If PGD does not move VGG-16, the attack is misconfigured.
- **AutoAttack is a lower bound:** AutoAttack robust accuracy must be $\leq$ PGD robust accuracy for the same model. A violation means a bug.
- **No gradient masking:** a black-box attack (Square) must not dramatically *outperform* a white-box attack (PGD) on any model. If it does, that model's apparent robustness is an artifact, not real.
- **Reproducibility:** with the fixed seed, clean accuracy per model must match published ImageNet numbers within $\sim 1\%$.
- **New architecture (Phase 4):** the defense model must beat its own base model on robust accuracy *and* the clean-accuracy delta must be reported honestly. Validate on a held-out attacked set not used during fine-tuning.

8. Constraints, gotchas, and pitfalls

- **Compute:** ~ 30 GPU-hours/week. AutoAttack is expensive; budget it. Start with the 1k-image subset; expand only if time allows.
- **Epsilon lives in $[0, 1]$ pixel space.** Never apply an attack after normalization, or `epsilon` becomes meaningless.
- **Fair comparison:** always report clean accuracy alongside robust accuracy.
- **Do not conflate protocols:** white-box and transfer results are separate tables.
- **Do not over-claim:** the defense module gives a *targeted*, *measured* improvement, not universal superiority.
- **Determinism:** set the seed for `torch`, `numpy`, and Python `random`; note that some CUDA ops remain non-deterministic.

9. Deliverables checklist

- [] Repository with the structure in Section 4, all phases implemented.
- [] `scripts/generate_datasets.py`, `scripts/run_benchmark.py`, `scripts/train_defense.py` runnable from the CLI.
- [] `results/`: ranking tables (white-box + transfer), heatmaps, tradeoff plot.
- [] A novel defense architecture with a measured robustness gain over its base.
- [] All Section 7 sanity checks passing.
- [] A workshop-paper draft following the Section 3.6 arc.

10. First actions for the agent

- 1 Initialize the repo and create the Section 4 directory skeleton.
- 2 Write `requirements.txt` and `config.yaml`.
- 3 Implement `models/base.py` and `models/classifiers.py`; smoke-test all 7 models on one clean image.
- 4 Proceed phase by phase (Section 6), running the Section 7 checks as you go.
- 5 When a design choice is ambiguous and this brief gives a recommended default, take the default and note the assumption.